IFT-6135 Assignment 1

By: Lucius Hatherly (20294880)

Problem 1: Implementation provided in mlp.py.

Problem 2:

Q1. Both discrete_2d_convolution and scipy.signal.convolve2d have the same theoretical time complexity of $O(HWK_HK_W)$. However, discrete_2d_convolution is significantly slower as it uses explicit Python loops where a slice is extracted at each iteration and small arrays are multiplied repeatedly. which require a high interpretative overhead. This is unlike scipy.signal.convolve2d which is compiled in optimized C code, where it uses optimized low-level loops, avoids Python interpretative overhead, and memory efficiency routines, which allows convolution operations to be executed more efficiently. Ultimately, scipy allows a much faster computation (20 to 100 x quicker) despite performing the same mathematical operation.

In addition, SciPy could also use highly optimized vector routines or FFT-based implementations internally for larger sized kernels. This additionally improved computational efficiency compared to a larger pure Python loop implementation.

Reference (scipy/scipy/signal at main · scipy/scipy, convolve2d — SciPy v1.17.0 Manual)

Q2.

Figure A: Images of Original Car and Car Image with Blur Kernel Applied

To blur the image we apply a 7x7 average (box) filter defined as:

$$K = \frac{1}{49}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Figure B: Image of Kernel used for Blurring

We can see that each output pixel is computed as the average of the 7x7 neighborhood. We have all coefficients equal and summing to 1, the filter is able to preserve overall brightness while smoothing local intensity variations.

The blurring effect occurs because the high-frequency details and sharp edges are averaged with the neighboring pixels. This causes intensity transitions to become smoother and reduces fine details. Essentially, increasing the kernel size increases the amount of smoothing which produces a stronger blurring impact.

Q3.

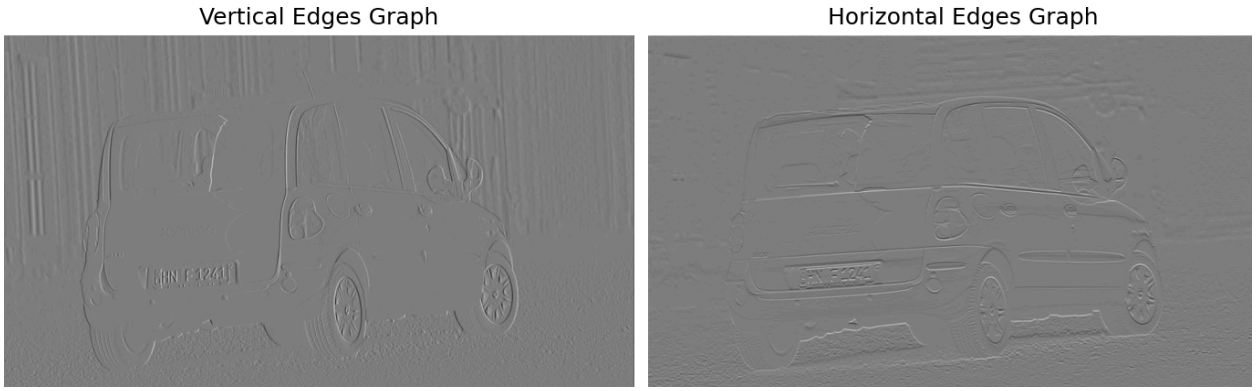

Figure C: Images of car with vertical and horizontal Kernel applied

In order to detect edges in the image, I applied two Sobel kernels using the discrete_2d_convolution function. This Sobel Kernel works through a gradient-based method that captures regions of strong intensity change, which corresponds to edges for these images.

Detection Kernel for Vertical Edges

$$K_v = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Figure D: Image of Vertical Kernel for Edge Detection

This kernel estimates the horizontal intensity gradient (∂I/∂x) which gives strong responses to vertical edges. This is because the vertical edges induce strong changes for intensity along the horizontal direction. We can see that the resulting vertical kernel applied image highlights the door frames, the wheels, background vertical structures, and the sides of the car. The flatter regions of the image remain close to zero even after the kernel is applied because we have minimal horizontal variation.

Detection Kernel for Horizontal Images:

$$K_h = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Figure E: Image of Horizontal Kernel for Edge Detection

The horizontal kernel estimate the vertical intensity gradient (∂I/∂y). This responds strongly to the horizontal edges, given that horizontal edges align with intensity changes across the vertical direction. We can see that the horizontal kernel edges highlight the car's bumper, roofline, edges of windows, and the boundary of the ground.

In order to understand why these Kernels detect edges we should recall that an edge corresponds to a sharp intensity change. The kernel convolutions perform a weighted difference across the neighboring pixels which include the negative weights detecting decreasing intensity, the central zero column (or row) separating the two regions being compared, and positive weights detecting increased intensity. Essentially, the Sobel kernel operator calculates a discrete gradient and the large absolute responses correspond to strong edges.

In conclusion, the kernel applied to the images reveals the vertical and horizontal edge maps highlight strong vertical and horizontal structures respectively. The regions with little texture appear darker while strong contours of the vehicles can be clearly visible in

both outputs. The results align with a theoretical understanding for Sobel edge detection, this confirms that the implementation of discrete_2d_convolution and selected kernels are correct.

Q4. Implementation provided in mobileNet.py

Q5. Implementation provided in mobileNet.py

Q6.

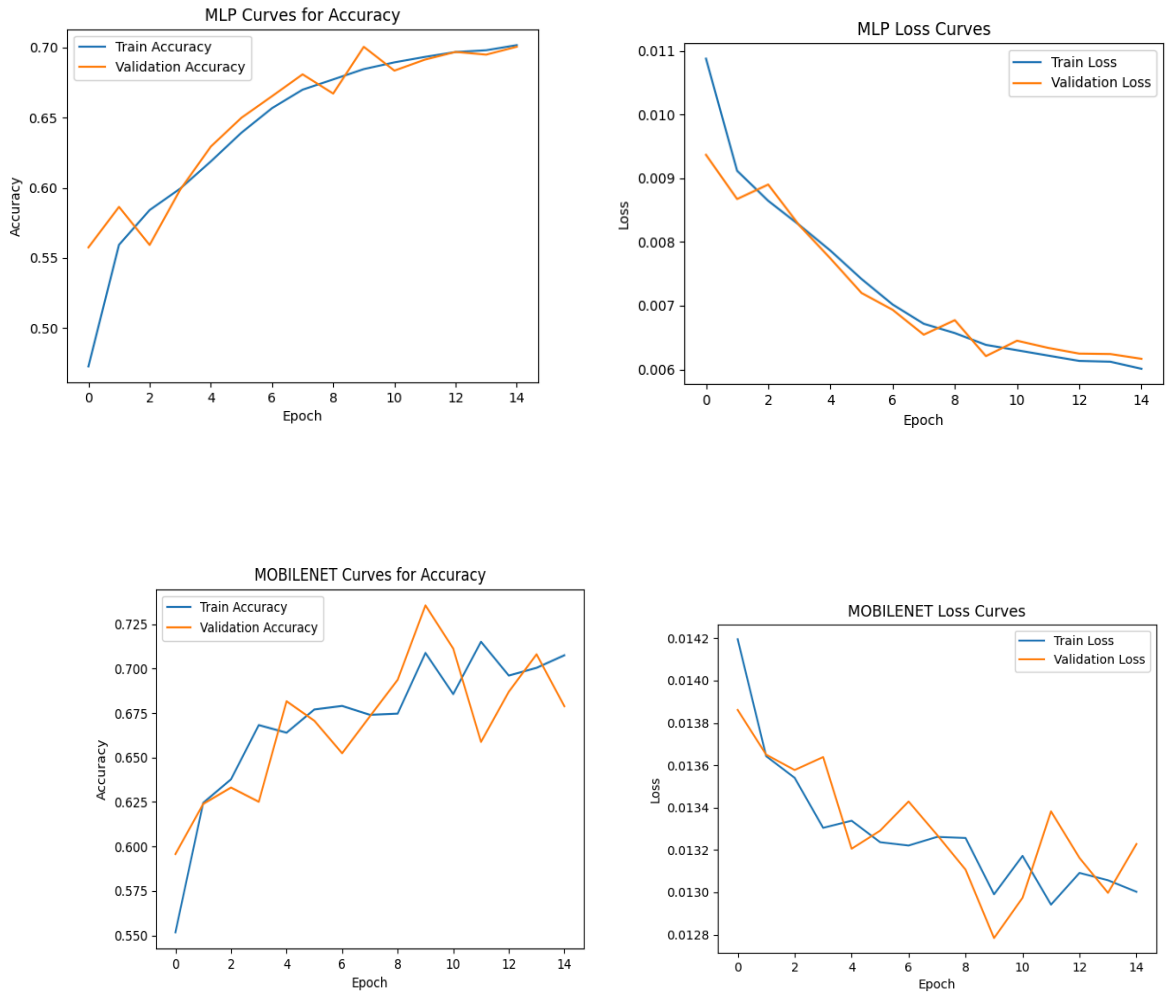

Figure F: Graphs for MLP and MOBILENET Curves depicting Accuracy and Loss

We can see from the curves that the two models both show gradual and overall improvement in training and validation accuracy across epochs with corresponding decreases in loss. However, MobileNet generally achieved higher validation and training accuracy compared to MLP. Specifically, MLP reaches approximately 70% validation accuracy while MobileNet surpasses this and reaches approximately 73% to 74%. Although it should be observed that MobileNet's training and validation accuracy starts

to decrease after the 10th epoch while MLP's training and validation accuracy consistently increases.

Moreover, the MobileNet accuracy curve further shows faster early improvement which indicates that the convolutional features are more efficient at extracting meaningful spatial patterns from the medical images. Since PathMNIST images contain structural and texture-based information, the convolutional layers in MobileNet are better suited to capture local spatial dependencies than a fully connected MLP, which flattens the image and ignores spatial structure. This architectural advantage explains why MobileNet achieves higher peak validation accuracy.

The loss curves for both models consistently decrease but the MLP loss curve decreases more smoothly than MOBILENET. Interestingly, despite the MOBILENET obtaining an overall higher accuracy than MLP, the loss is overall greater than the loss of MLP. This difference is derived from accuracy and loss measuring different aspects of performance. The accuracy only reflects whether the predicted class is correct, while cross-entropy loss further decreases the confidence of the predictions. The MobileNet may correctly classify more samples (thus achieving higher accuracy) but it may also produce more confident incorrect predictions on certain examples. This serves to increase the average loss. This contrasts with MLP producing less confident predictions which results in a marginally lower loss even if the classification accuracy itself is lower.

Moreover, the slight decrease and fluctuation in MobileNet's validation accuracy after around the 10th epoch implies the start of mild overfitting. Due to its higher representational capacity along with convolutional feature extraction, the MobileNet model can fit the training data more aggressively. After the 10th epoch, the model may begin to specialize too strongly to patterns only observed within the training data, which in turn diminishes the generalization performance on the validation set. The MLP is a simpler model with a lower capacity, and appears to generalize more steadily, which results in smoother training dynamics. In conclusion, the MobileNet demonstrates a better feature extraction capability, however, the higher capacity makes it more sensitive towards overfitting for later epochs.

Problem 3:

Q1. Implementation provided in unet.py

Q2. Implementation provided in utils.py

Problem 4:

Q1.

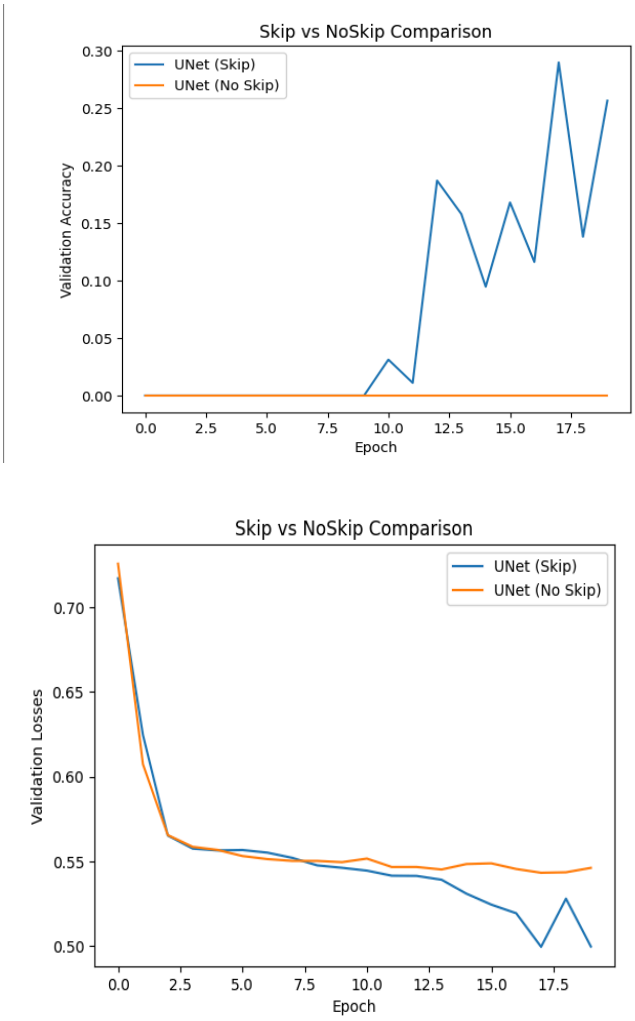

Figure G: Graphs showing performance of Skip vs NoSkip for UNet Model

For this question, two models were trained: the standard UNet architecture and a modified UNet without skip connections (implemented in unet). The two models share in common the same encoder–decoder structure, however, the NoSkip variant removes the concatenation of encoder feature maps into the decoder pathway.

We can see from the validation accuracy plot, we can see a drastic performance difference between the two architectures. The standard UNet (with skip connections) has steadily improving validation Dice accuracy across the epochs, reaching approximately 0.25–0.30 by the end of training. This is contrasted with the UNet without skip connections where we see approximately 0 validation accuracy across all epochs. This implies that the NoSkip model struggles to properly segment the retinal blood vessels.

The validation loss curves further reinforce this observation. The two models both show the validation loss decreasing initially, the UNet with skip connections achieves consistently lower loss values and it further continues improving over the various epochs. The NoSkip model loss graph also plateaus at a higher loss and further illustrates minimal improvement, this further confirms that the NoSkip model fails to learn meaningful segmentation features.

This behavior is not surprising and even expected theoretically. The skip connections in UNet permits the direct transfer of high-resolution spatial features from the encoder to the decoder. During segmentation, we can note that the exact localization is imperative, especially for thin structures like retinal vessels. If we remove the skip connections, the decoder only focuses on deeply compressed bottleneck features (post multiple pooling operations), which lose granular fine-grained spatial information. This creates a strong information bottleneck which inhibits the decoder's ability to obtain precise boundary details required for vessel segmentation. Consequently, the NoSkip model fails to accurately reconstruct detailed vessel boundaries.

In conclusion, we can see that we need skip connections in order to have effective semantic segmentation in UNet. Skip connections drastically improve both quantitative performance (Dice accuracy and loss) and the model's ability to extract granular fine spatial details.

Q2.

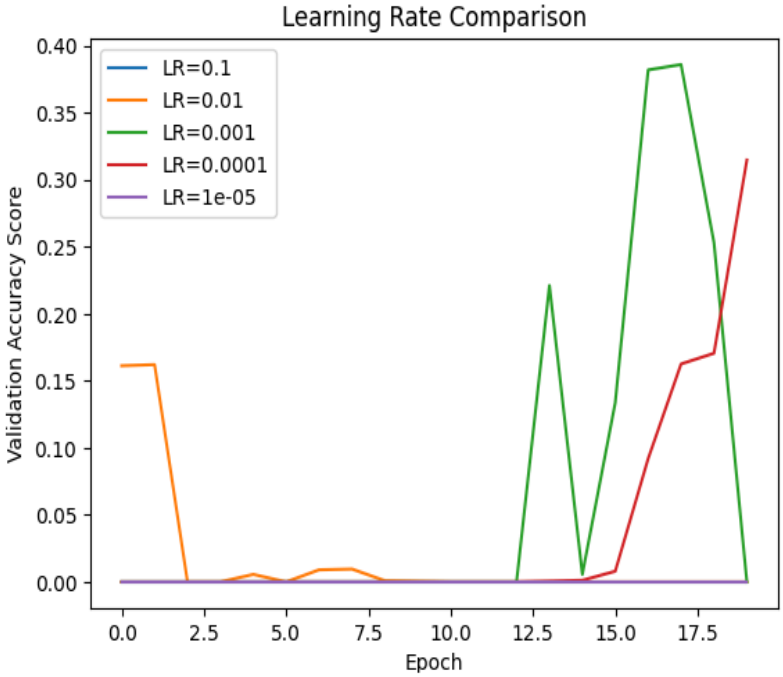

Figure H: Learning Rate Accuracy Graph

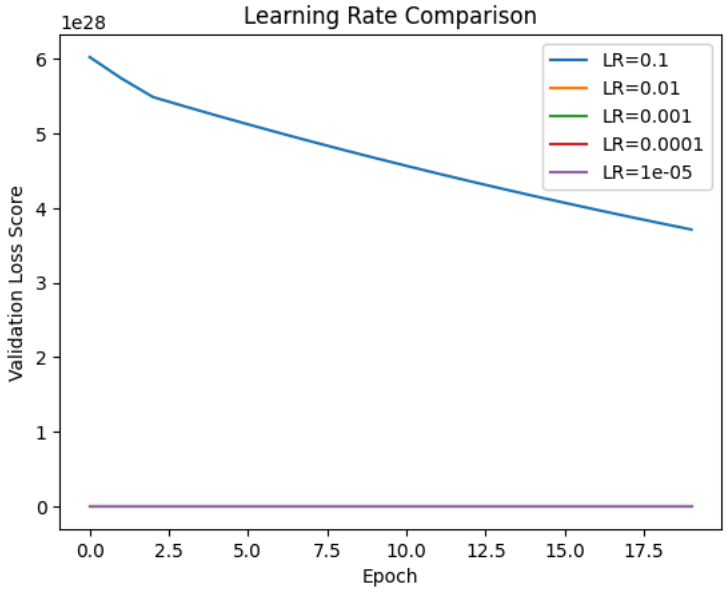

Figure I: Validation Loss Graph

```
==============================
Training with Learning Rate: 0.1
==============================
[TRAIN] Epoch: 0, Iter: 0, Loss: 0.76394
== [TRAIN] Epoch: 0, Accuracy: 0.000 ==>
[VAL] Epoch: 0, Iter: 0, Loss: 7156641568253027615232543948 8.00000
=== [VAL] Epoch: 0, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 1, Iter: 0, Loss: 649890328096921919254011445248.00000
== [TRAIN] Epoch: 1, Accuracy: 0.000 ==>
[VAL] Epoch: 1, Iter: 0, Loss: 6946391423229408697099327897 6.00000
=== [VAL] Epoch: 1, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 2, Iter: 0, Loss: 591559185063867767896020090 88.00000
== [TRAIN] Epoch: 2, Accuracy: 0.000 ==>
[VAL] Epoch: 2, Iter: 0, Loss: 678818175684701326922104176 64.00000
=== [VAL] Epoch: 2, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 3, Iter: 0, Loss: 613761154964755261385951150 08.00000
== [TRAIN] Epoch: 3, Accuracy: 0.000 ==>
[VAL] Epoch: 3, Iter: 0, Loss: 663385104229364696792128552 96.00000
=== [VAL] Epoch: 3, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 4, Iter: 0, Loss: 455286752717330161196985221 12.00000
== [TRAIN] Epoch: 4, Accuracy: 0.000 ==>
[VAL] Epoch: 4, Iter: 0, Loss: 648274098168159776465726996 48.00000
=== [VAL] Epoch: 4, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 5, Iter: 0, Loss: 574353903914839817310248632 32.00000
== [TRAIN] Epoch: 5, Accuracy: 0.000 ==>
[VAL] Epoch: 5, Iter: 0, Loss: 633601044374576187036443607 04.00000
=== [VAL] Epoch: 5, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 6, Iter: 0, Loss: 565881883997242016403973734 40.00000
== [TRAIN] Epoch: 6, Accuracy: 0.000 ==>
[VAL] Epoch: 6, Iter: 0, Loss: 619126424420602780099039723 52.00000
=== [VAL] Epoch: 6, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 7, Iter: 0, Loss: 497660783286443631181740113 92.00000
== [TRAIN] Epoch: 7, Accuracy: 0.000 ==>
[VAL] Epoch: 7, Iter: 0, Loss: 605056700168870616542258135 04.00000
=== [VAL] Epoch: 7, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 8, Iter: 0, Loss: 534426578644165939207161774 08.00000
== [TRAIN] Epoch: 8, Accuracy: 0.000 ==>
[VAL] Epoch: 8, Iter: 0, Loss: 591328119671860956155713945 60.00000
=== [VAL] Epoch: 8, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 9, Iter: 0, Loss: 532022374644072174132417003 52.00000
== [TRAIN] Epoch: 9, Accuracy: 0.000 ==>
[VAL] Epoch: 9, Iter: 0, Loss: 577855963674871117504273448 96.00000
=== [VAL] Epoch: 9, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 10, Iter: 0, Loss: 437766017487246518310938869 76.00000
== [TRAIN] Epoch: 10, Accuracy: 0.000 ==>
[VAL] Epoch: 10, Iter: 0, Loss: 564721173539417486306899394 56.00000
=== [VAL] Epoch: 10, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 11, Iter: 0, Loss: 536174373703140823597202800 64.00000
== [TRAIN] Epoch: 11, Accuracy: 0.000 ==>
[VAL] Epoch: 11, Iter: 0, Loss: 551888378740543368920941199 36.00000
=== [VAL] Epoch: 11, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 12, Iter: 0, Loss: 507779728514942220856291491 84.00000
```

Figure J: 0.1 Learning Rate Loss Scores

```
==============================
Training with Learning Rate: 0.01
==============================
[TRAIN] Epoch: 0, Iter: 0, Loss: 0.78512
== [TRAIN] Epoch: 0, Accuracy: 0.074 ==>
[VAL] Epoch: 0, Iter: 0, Loss: 137666.85938
=== [VAL] Epoch: 0, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 1, Iter: 0, Loss: 132037.03125
== [TRAIN] Epoch: 1, Accuracy: 0.004 ==>
[VAL] Epoch: 1, Iter: 0, Loss: 1528373120.00000
=== [VAL] Epoch: 1, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 2, Iter: 0, Loss: 7009434624.00000
== [TRAIN] Epoch: 2, Accuracy: 0.016 ==>
[VAL] Epoch: 2, Iter: 0, Loss: 0.65618
=== [VAL] Epoch: 2, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 3, Iter: 0, Loss: 0.65773
== [TRAIN] Epoch: 3, Accuracy: 0.000 ==>
[VAL] Epoch: 3, Iter: 0, Loss: 0.63519
=== [VAL] Epoch: 3, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 4, Iter: 0, Loss: 0.63449
== [TRAIN] Epoch: 4, Accuracy: 0.000 ==>
[VAL] Epoch: 4, Iter: 0, Loss: 0.62146
=== [VAL] Epoch: 4, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 5, Iter: 0, Loss: 0.62157
== [TRAIN] Epoch: 5, Accuracy: 0.000 ==>
[VAL] Epoch: 5, Iter: 0, Loss: 0.61335
=== [VAL] Epoch: 5, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 6, Iter: 0, Loss: 0.61303
== [TRAIN] Epoch: 6, Accuracy: 0.000 ==>
[VAL] Epoch: 6, Iter: 0, Loss: 0.60879
=== [VAL] Epoch: 6, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 7, Iter: 0, Loss: 0.60881
== [TRAIN] Epoch: 7, Accuracy: 0.000 ==>
[VAL] Epoch: 7, Iter: 0, Loss: 0.60614
=== [VAL] Epoch: 7, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 8, Iter: 0, Loss: 0.60034
== [TRAIN] Epoch: 8, Accuracy: 0.000 ==>
[VAL] Epoch: 8, Iter: 0, Loss: 0.60465
=== [VAL] Epoch: 8, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 9, Iter: 0, Loss: 0.60600
== [TRAIN] Epoch: 9, Accuracy: 0.000 ==>
[VAL] Epoch: 9, Iter: 0, Loss: 0.60384
=== [VAL] Epoch: 9, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 10, Iter: 0, Loss: 0.59695
== [TRAIN] Epoch: 10, Accuracy: 0.000 ==>
[VAL] Epoch: 10, Iter: 0, Loss: 0.60342
=== [VAL] Epoch: 10, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 11, Iter: 0, Loss: 0.60495
== [TRAIN] Epoch: 11, Accuracy: 0.000 ==>
[VAL] Epoch: 11, Iter: 0, Loss: 0.60323
=== [VAL] Epoch: 11, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 12, Iter: 0, Loss: 0.59494
```

Figure K: 0.01 Learning Rates Lost Score

```
============================
Training with Learning Rate: 0.001
============================
[TRAIN] Epoch: 0, Iter: 0, Loss: 0.77089
== [TRAIN] Epoch: 0, Accuracy: 0.009 ==>
[VAL] Epoch: 0, Iter: 0, Loss: 0.62017
=== [VAL] Epoch: 0, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 1, Iter: 0, Loss: 0.60183
== [TRAIN] Epoch: 1, Accuracy: 0.000 ==>
[VAL] Epoch: 1, Iter: 0, Loss: 0.55683
=== [VAL] Epoch: 1, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 2, Iter: 0, Loss: 0.56378
== [TRAIN] Epoch: 2, Accuracy: 0.000 ==>
[VAL] Epoch: 2, Iter: 0, Loss: 0.55347
=== [VAL] Epoch: 2, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 3, Iter: 0, Loss: 0.55806
== [TRAIN] Epoch: 3, Accuracy: 0.000 ==>
[VAL] Epoch: 3, Iter: 0, Loss: 0.55132
=== [VAL] Epoch: 3, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 4, Iter: 0, Loss: 0.55293
== [TRAIN] Epoch: 4, Accuracy: 0.000 ==>
[VAL] Epoch: 4, Iter: 0, Loss: 0.55001
=== [VAL] Epoch: 4, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 5, Iter: 0, Loss: 0.55024
== [TRAIN] Epoch: 5, Accuracy: 0.000 ==>
[VAL] Epoch: 5, Iter: 0, Loss: 0.54474
=== [VAL] Epoch: 5, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 6, Iter: 0, Loss: 0.55093
== [TRAIN] Epoch: 6, Accuracy: 0.000 ==>
[VAL] Epoch: 6, Iter: 0, Loss: 0.54884
=== [VAL] Epoch: 6, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 7, Iter: 0, Loss: 0.54856
== [TRAIN] Epoch: 7, Accuracy: 0.000 ==>
[VAL] Epoch: 7, Iter: 0, Loss: 0.53867
=== [VAL] Epoch: 7, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 8, Iter: 0, Loss: 0.53877
== [TRAIN] Epoch: 8, Accuracy: 0.000 ==>
[VAL] Epoch: 8, Iter: 0, Loss: 0.55560
=== [VAL] Epoch: 8, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 9, Iter: 0, Loss: 0.55535
== [TRAIN] Epoch: 9, Accuracy: 0.000 ==>
[VAL] Epoch: 9, Iter: 0, Loss: 0.53881
=== [VAL] Epoch: 9, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 10, Iter: 0, Loss: 0.53948
== [TRAIN] Epoch: 10, Accuracy: 0.000 ==>
[VAL] Epoch: 10, Iter: 0, Loss: 0.55082
=== [VAL] Epoch: 10, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 11, Iter: 0, Loss: 0.53660
== [TRAIN] Epoch: 11, Accuracy: 0.000 ==>
[VAL] Epoch: 11, Iter: 0, Loss: 0.53237
=== [VAL] Epoch: 11, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 12, Iter: 0, Loss: 0.53757
== [TRAIN] Epoch: 12, Accuracy: 0.000 ==>
```

Figure L: 0.001 Learning Rates Lost Score

```
==============================
Training with Learning Rate: 0.0001
==============================
[TRAIN] Epoch: 0, Iter: 0, Loss: 0.79631
== [TRAIN] Epoch: 0, Accuracy: 0.096 ==>
[VAL] Epoch: 0, Iter: 0, Loss: 0.73385
=== [VAL] Epoch: 0, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 1, Iter: 0, Loss: 0.73535
== [TRAIN] Epoch: 1, Accuracy: 0.000 ==>
[VAL] Epoch: 1, Iter: 0, Loss: 0.60759
=== [VAL] Epoch: 1, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 2, Iter: 0, Loss: 0.60310
== [TRAIN] Epoch: 2, Accuracy: 0.000 ==>
[VAL] Epoch: 2, Iter: 0, Loss: 0.56057
=== [VAL] Epoch: 2, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 3, Iter: 0, Loss: 0.56524
== [TRAIN] Epoch: 3, Accuracy: 0.000 ==>
[VAL] Epoch: 3, Iter: 0, Loss: 0.55928
=== [VAL] Epoch: 3, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 4, Iter: 0, Loss: 0.55764
== [TRAIN] Epoch: 4, Accuracy: 0.000 ==>
[VAL] Epoch: 4, Iter: 0, Loss: 0.55910
=== [VAL] Epoch: 4, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 5, Iter: 0, Loss: 0.55792
== [TRAIN] Epoch: 5, Accuracy: 0.000 ==>
[VAL] Epoch: 5, Iter: 0, Loss: 0.55874
=== [VAL] Epoch: 5, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 6, Iter: 0, Loss: 0.55666
== [TRAIN] Epoch: 6, Accuracy: 0.000 ==>
[VAL] Epoch: 6, Iter: 0, Loss: 0.55768
=== [VAL] Epoch: 6, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 7, Iter: 0, Loss: 0.56096
== [TRAIN] Epoch: 7, Accuracy: 0.000 ==>
[VAL] Epoch: 7, Iter: 0, Loss: 0.55778
=== [VAL] Epoch: 7, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 8, Iter: 0, Loss: 0.55644
== [TRAIN] Epoch: 8, Accuracy: 0.000 ==>
[VAL] Epoch: 8, Iter: 0, Loss: 0.55707
=== [VAL] Epoch: 8, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 9, Iter: 0, Loss: 0.55346
== [TRAIN] Epoch: 9, Accuracy: 0.000 ==>
[VAL] Epoch: 9, Iter: 0, Loss: 0.55550
=== [VAL] Epoch: 9, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 10, Iter: 0, Loss: 0.54417
== [TRAIN] Epoch: 10, Accuracy: 0.002 ==>
[VAL] Epoch: 10, Iter: 0, Loss: 0.55140
=== [VAL] Epoch: 10, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 11, Iter: 0, Loss: 0.54956
== [TRAIN] Epoch: 11, Accuracy: 0.000 ==>
[VAL] Epoch: 11, Iter: 0, Loss: 0.54996
=== [VAL] Epoch: 11, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 12, Iter: 0, Loss: 0.55402
```

Figure M: 0.0001 Learning Rates Lost Score

```
============================
Training with Learning Rate: 1e-05
============================
[TRAIN] Epoch: 0, Iter: 0, Loss: 0.75937
== [TRAIN] Epoch: 0, Accuracy: 0.000 ==>
[VAL] Epoch: 0, Iter: 0, Loss: 0.74545
=== [VAL] Epoch: 0, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 1, Iter: 0, Loss: 0.75451
== [TRAIN] Epoch: 1, Accuracy: 0.000 ==>
[VAL] Epoch: 1, Iter: 0, Loss: 0.74197
=== [VAL] Epoch: 1, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 2, Iter: 0, Loss: 0.74852
== [TRAIN] Epoch: 2, Accuracy: 0.000 ==>
[VAL] Epoch: 2, Iter: 0, Loss: 0.73695
=== [VAL] Epoch: 2, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 3, Iter: 0, Loss: 0.74177
== [TRAIN] Epoch: 3, Accuracy: 0.000 ==>
[VAL] Epoch: 3, Iter: 0, Loss: 0.72920
=== [VAL] Epoch: 3, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 4, Iter: 0, Loss: 0.73039
== [TRAIN] Epoch: 4, Accuracy: 0.000 ==>
[VAL] Epoch: 4, Iter: 0, Loss: 0.71532
=== [VAL] Epoch: 4, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 5, Iter: 0, Loss: 0.71189
== [TRAIN] Epoch: 5, Accuracy: 0.000 ==>
[VAL] Epoch: 5, Iter: 0, Loss: 0.65679
=== [VAL] Epoch: 5, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 6, Iter: 0, Loss: 0.65718
== [TRAIN] Epoch: 6, Accuracy: 0.000 ==>
[VAL] Epoch: 6, Iter: 0, Loss: 0.64670
=== [VAL] Epoch: 6, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 7, Iter: 0, Loss: 0.64071
== [TRAIN] Epoch: 7, Accuracy: 0.000 ==>
[VAL] Epoch: 7, Iter: 0, Loss: 0.63560
=== [VAL] Epoch: 7, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 8, Iter: 0, Loss: 0.62819
== [TRAIN] Epoch: 8, Accuracy: 0.000 ==>
[VAL] Epoch: 8, Iter: 0, Loss: 0.62085
=== [VAL] Epoch: 8, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 9, Iter: 0, Loss: 0.61259
== [TRAIN] Epoch: 9, Accuracy: 0.000 ==>
[VAL] Epoch: 9, Iter: 0, Loss: 0.59178
=== [VAL] Epoch: 9, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 10, Iter: 0, Loss: 0.59310
== [TRAIN] Epoch: 10, Accuracy: 0.000 ==>
[VAL] Epoch: 10, Iter: 0, Loss: 0.56799
=== [VAL] Epoch: 10, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 11, Iter: 0, Loss: 0.56525
== [TRAIN] Epoch: 11, Accuracy: 0.000 ==>
[VAL] Epoch: 11, Iter: 0, Loss: 0.56571
=== [VAL] Epoch: 11, Iter: 4, Accuracy: 0.000 ===>
[TRAIN] Epoch: 12, Iter: 0, Loss: 0.57412
```

Figure N: 0.00001 Learning Rates Lost Score

To investigate the effect of the learning rate when training the UNet model with AdamW, experiments were conducted using learning rates of 0.1, 0.01, 0.001, 0.0001, and 0.00001. Both the two curves of validation accuracy and validation loss curves show a strong sensitivity of training stability and convergence to the choice of learning rate.

We can see from the validation accuracy graph, the very large learning rates (0.1 and 0.01) perform particularly poorly. Specifically, LR = 0.1 results in almost zero validation accuracy across all epochs,this indicates that the model fails to learn significant segmentation features. When LR = 0.01 shows gradual initial improvements but rapidly collapses to around 0. This suggests unstable optimization where the parameter updates are far too large and as a result overshoot the minima of the particular loss function.

Carrying on, LR = 0.001 produces the best performance in validation accuracy. However, the accuracy curve demonstrates oscillations in later epochs, it reaches the highest validation Dice score (approximately 0.38–0.39). These facts imply that the learning rate is large enough to make significant progress while keeping sufficient stability for convergence. We can further see that LR = 0.0001 also improves gradually and reaches moderate performance (around 0.30), but converges more gradually. The smallest learning rate, 0.00001, generates virtually no improvement for validation accuracy. This suggests the updates are too miniscule for effective learning within the assigned epoch number.

In addition, the validation loss graph also explains the trends, when we have LR = 0.1, the validation loss is seen to explode to considerably large values (for the order of 10^{28}). This drastic divergence dominates the plot's scale which explains why only two curves are clearly visible which are the exploding LR = 0.1 curve and the essentially 0 or less flat curves for the other learning rates. We can see that the LR = 0.1 loss is so large, it effectively compresses the y-axis, which makes the smaller loss curves indistinguishable from one another. This further confirms that LR = 0.1 produces numerical instability and divergence during optimization especially compared to the other learning rates.

In addition, we can see from the numerical results of the loss for the remaining learning rates, the validation loss values remain small and stable and consistently decrease across the epochs.

In conclusion, the graphs shows three key trends:

1. **Too large (0.1, 0.01):** unstable training, ineffective or unstable loss, poor accuracy.

2. **Well-balanced (0.001):** more stable convergence, decreasing loss, and it demonstrates the best validation accuracy.

3. **Too small (0.00001):** stable but very excessively slow optimization, and there is limited accuracy improvement.

In conclusion, we can see among the tested values, the LR = 0.001 gives the optimal trade-off between convergence speed and training stability for this given segmentation task.

Q3.

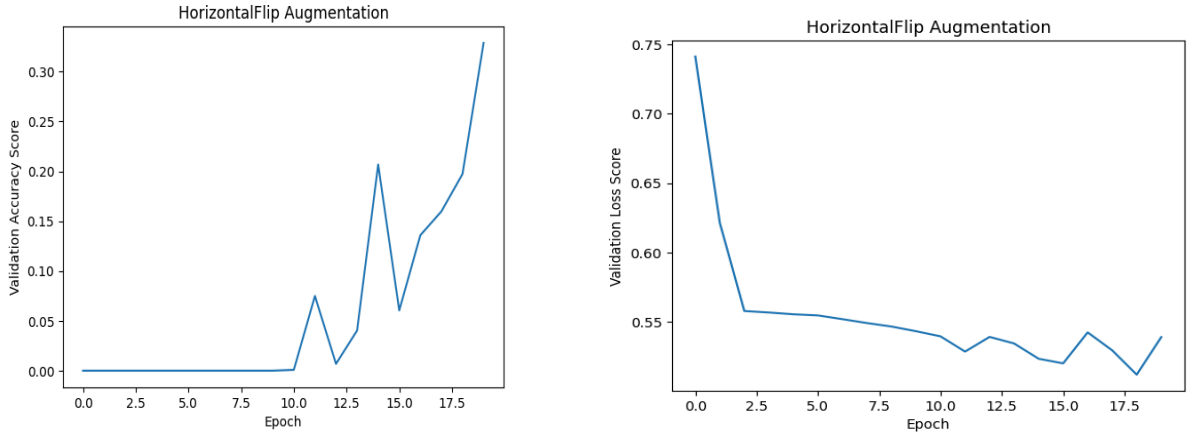

Figure O: HorizontalFlip Augmentation Graphs

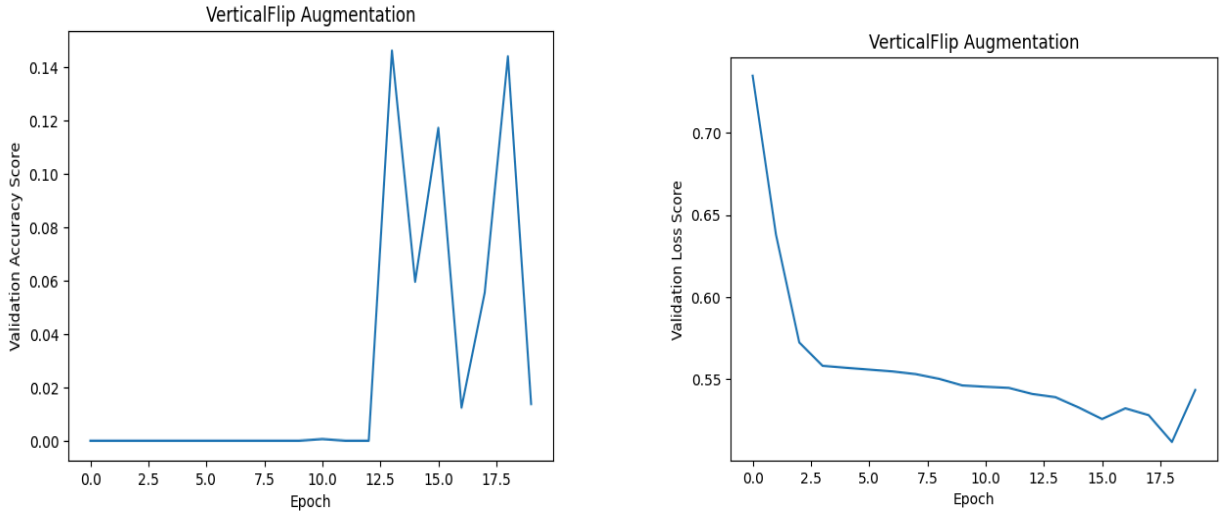

Figure P: VerticalFlip Augmentation Graphs

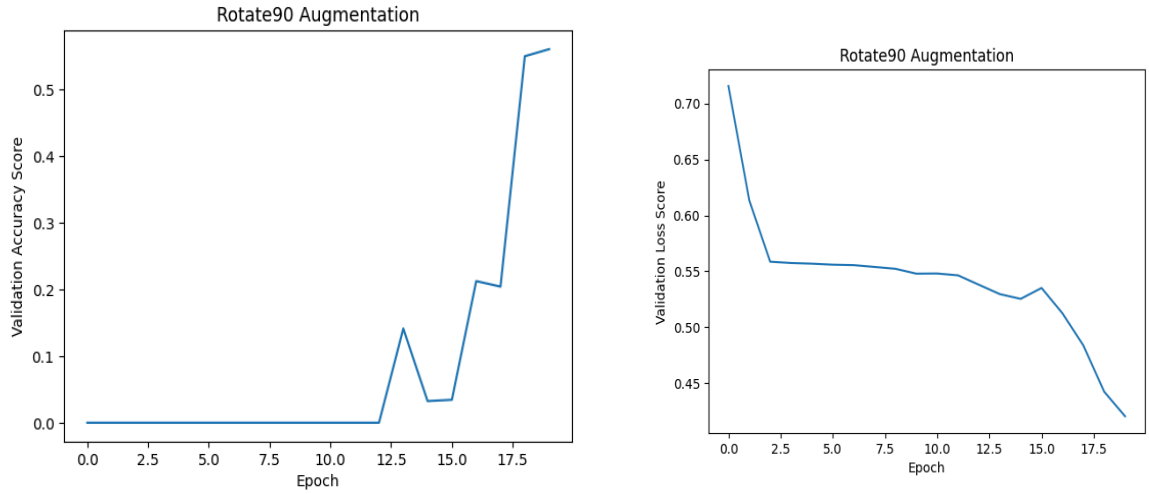

Figure Q: Rotate90 Augmentation Graphs

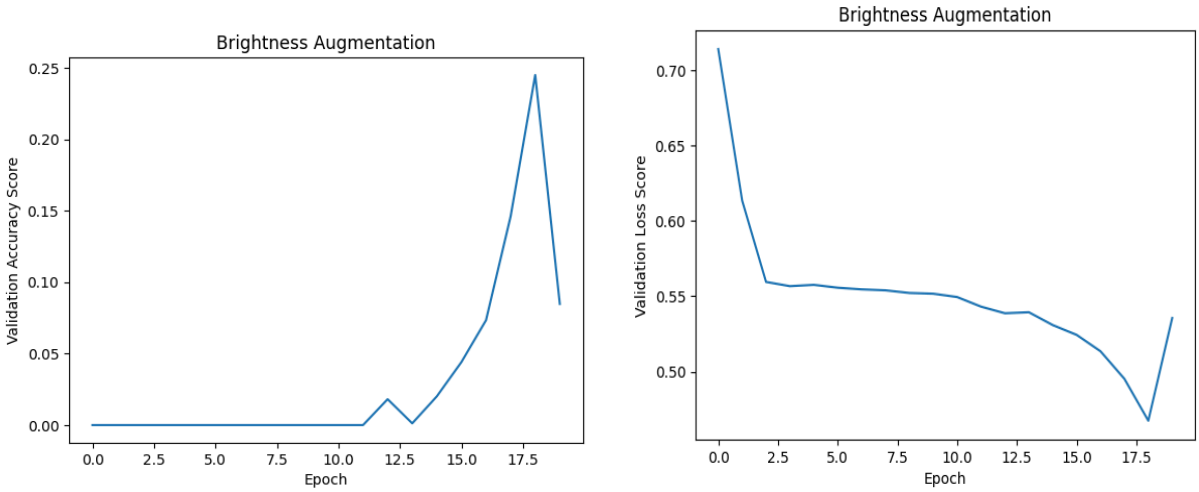

Figure R: Brightness Augmentation Graphs

In order to investigate the impact of data augmentation, four augmentation strategies were examined: 1. random horizontal flip, 2. random vertical flip, 3. random 90-degree rotation, and 4. random brightness adjustment. Across each augmentation, a UNet model was trained and then validation Dice accuracy and validation loss was measured across each epoch.

For the validation accuracy curves, it can be seen that **Rotate90 augmentation produced the strongest improvement**, reaching the highest validation Dice score across the four data augmentation methods (around 0.55 toward the final epochs). This

implies that Rotate90 is able to significantly improve generalization. It should be noted that in the retinal vessel segmentation, vessels can appear in many orientations, and 90-degree rotations help the model learn orientation-invariant features. The validation loss curve for Rotate90 further shows a downward trend that consistently declines. This further reconfirms a more stable learning and improved segmentation performance.

The **HorizontalFlip augmentation** demonstrates moderate improvement by reaching a validation Dice score of around 0.32. The validation loss decreased gradually, indicating that horizontal flipping introduces useful variability without destabilizing training. The retinal images are generally symmetric to some extent, horizontal flips help increase data diversity while preserving anatomical realism. However, it should be noted that HorizontalFlip is unable to generalize as well as the Rotate90 augmentation.

The **Brightness augmentation** resulted in smaller improvements in Dice score compared to HorizontalFlip and Random Rotate (peaking around 0.25). Although the validation loss decreased steadily, the accuracy curve remained unstable. This implies that although brightness variation helps the model become marginally more robust to illumination differences, the intensity changes are insufficient to significantly alter the vessel structure, which notably is the main target for the segmentation.

The **VerticalFlip augmentation** performed the poorest compared to all other data augmentation methods. The graph shows highly unstable and low validation Dice scores (around 0.14 as its peak score). Although the validation loss decreased as the graph shows, the segmentation accuracy remains low. This is possibly due to vertical flipping causing a distortion of anatomical realism in retinal images, as vessels adhere to organized patterns that are likely not vertically symmetric for the given dataset.

In conclusion, we can see that the **rotation augmentation was the most effective strategy**. This is followed by horizontal flipping, brightness adjustment, and finally vertical flipping. The graphs and results as a whole imply that geometric augmentations that preserve anatomical structure but introduce diversity in orientation are the most advantageous for this given segmentation task. This is unlike augmentations which modify natural structure (like vertical flipping) or only change pixel intensity (brightness) essentially only induce limited performance gains.

Q4.

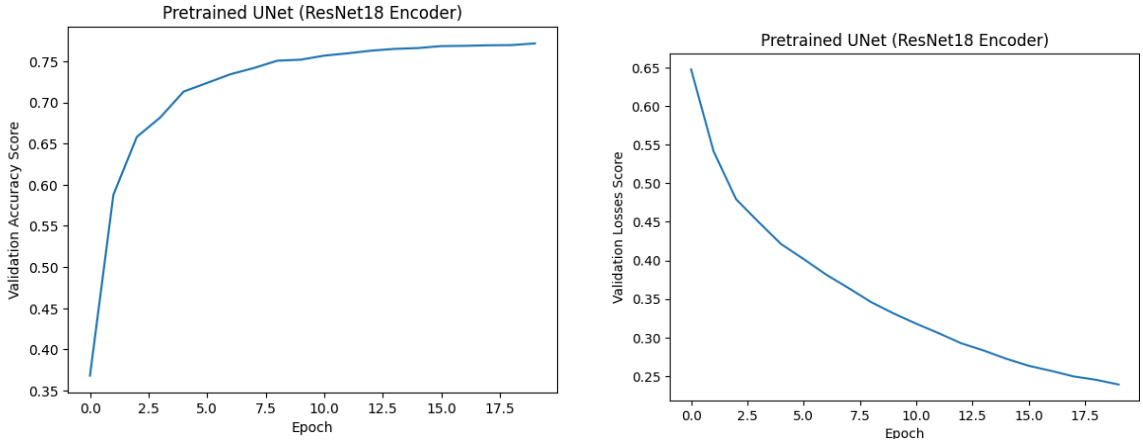

Figure P: UNet Graphs

In order to evaluate the impact of transfer learning, a pretrained UNet with a ResNet18 encoder initialized with ImageNet weights was fine-tuned and examined against the UNet trained from scratch (NoSkip version). The results show that the pretrained UNet model outperformed the UNet NoSkip. The pretrained UNet generates a validation Dice score of around 0.77 after 20 epochs, whereas the UNet trained from scratch (NoSkip) remains significantly lower, it only achieves around 0.25. This large performance discrepancy indicates the efficacy of transfer learning for medical image segmentation tasks like the retinal blood vessels.

The pretrained model further shows faster convergence. The validation Dice score increases drastically within the first few epochs, while the validation loss gradually decreases and smoothly from around 0.65 to around 0.24. This is in contrast to the UNet trained from scratch illustrates more gradual and slower improvement, lower final accuracy, a higher loss, and generally less stable behavior. Although the validation loss decreases gradually, it flattens at a higher value (around 0.50–0.55). This illustrates reduced learning capacity under similar training conditions.

The superior performance of the pretrained UNet can be explained by the fact that the ResNet18 encoder has already learned more advanced low-level and mid-level visual features (like textures, edges and shapes) from large-scale natural image datasets. These given learned representations can be properly transferred to retinal vessel segmentation, permitting the models to concentrate on learning task-specific features compared to starting from a random initialization. This is unlike the model trained from

the onset where we must learn all feature representations from limited medical data. This then causes a slower convergence and less accurate final performance.

In conclusion, the results show that fine-tuning a pretrained encoder definitely improves segmentation accuracy, diminishes validation loss, and improves generalization compared to training a given UNet from scratch. The transfer learning is able to provide a strong benefit for medical image segmentation tasks where the given labeled data is limited.